

## CO1-AT3-Coding Challenge

View Only

[← Back](#)

## QUESTIONS

## Q1

Smart University Payroll  
Management System  
25 marks

## Q2

Smart Vehicle Insurance Claim  
System  
25 marks

2/2 answered

## Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary:  $\text{basicSalary} + (\text{teachingHours} \times \text{incentivePerHour})$  TechnicalStaff overtimeHours overtimeRate Salary:  $\text{basicSalary} + (\text{overtimeHours} \times \text{overtimeRate})$  VisitingFaculty lecturesHandled paymentPerLecture Salary:  $\text{lecturesHandled} \times \text{paymentPerLecture}$  Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty

101

Dr. Arun

CSE

50000

20

500

TechnicalStaff

102

Mr. Ravi

ECE

35000

15

300

VisitingFaculty

103

Dr. Kumar

IT

0

40

1200

Search Employee ID:

102

Expected Output

Employee Details

Employee ID : 101

Name : Dr. Arun

Department : CSE

Monthly Salary : 60000.0

Employee ID : 102

Name : Mr. Ravi

Department : ECE

Monthly Salary : 39500.0

## Your Code

```
1 import java.util.*;
2
3 class Main
4 {
5     public static String readInput(Scanner sc)
6     {
7         String s = "";
8
9         while(sc.hasNextLine())
10        {
11            s = sc.nextLine().trim();
12
13            if(!s.isEmpty())
14            {
15                return s;
16            }
17        }
18
19        return s;
20    }
21
22    public static void main(String args[])
23    {
24        Scanner sc = new Scanner(System.in);
25
26        int n = Integer.parseInt(readInput(sc));
27
28        Employee emp[] = new Employee[n];
29
30        for(int i=0;i<n;i++)
31        {
32            String type = readInput(sc);
33
34            int id = Integer.parseInt(readInput(sc));
35            String name = readInput(sc);
36            String department = readInput(sc);
37            double basicSalary = Double.parseDouble(readInput(sc));
38
39            if(type.equalsIgnoreCase("TeachingFaculty"))
40            {
41                int teachingHours = Integer.parseInt(readInput(sc));
42                double incentivePerHour = Double.parseDouble(readInput(sc));
43
44                emp[i] = new TeachingFaculty(id,name,department,
45                    basicSalary,teachingHours,incentivePerHour);
46            }
47            else if(type.equalsIgnoreCase("TechnicalStaff"))
48            {
49                int overtimeHours = Integer.parseInt(readInput(sc));
50                double overtimeRate = Double.parseDouble(readInput(sc));
51
52                emp[i] = new TechnicalStaff(id,name,department,
53                    basicSalary,overtimeHours,overtimeRate);
54            }
55            else if(type.equalsIgnoreCase("VisitingFaculty"))
56            {
57                int lecturesHandled = Integer.parseInt(readInput(sc));
58                double paymentPerLecture = Double.parseDouble(readInput(sc));
59            }
```

## Input

```
3
TeachingFaculty
101
Dr. Arun
```

## Output

```
Employee Details
Employee ID : 101
Name : Dr. Arun
Department : CSE
Monthly Salary : 60000.0
```

```
62     }
63 }
64
65 String search = readInput(sc);
66
67 if(search.equalsIgnoreCase("Search Employee ID:"))
68 {
69     search = readInput(sc);
70 }
71
72 int searchId = Integer.parseInt(search);
73
74 Employee highest = emp[0];
75 double totalSalary = 0;
76
77 System.out.println("Employee Details");
78 System.out.println();
79
80 for(int i=0;i<n;i++)
81 {
82     emp[i].displayEmployee();
83
84     System.out.println("Monthly Salary : " + emp[i].calculateSalary());
85     System.out.println();
86
87     totalSalary += emp[i].calculateSalary();
88
89     if(emp[i].calculateSalary() > highest.calculateSalary())
90     {
91         highest = emp[i];
92     }
93 }
94
95 System.out.println("Highest Paid Employee");
96 System.out.println("Employee ID : " + highest.getEmployeeId());
97 System.out.println("Salary : " + highest.calculateSalary());
98 System.out.println();
99
100 System.out.println("Average Salary");
101
102 double avg = totalSalary / n;
103
104 if(avg == Math.floor(avg))
105     System.out.println(avg);
106 else
107     System.out.printf("%.2f\n",avg);
108
109 System.out.println();
110
111 boolean found = false;
112
113 for(int i=0;i<n;i++)
114 {
115     if(emp[i].getEmployeeId()==searchId)
116     {
117         System.out.println("Employee Found");
118         System.out.println("Employee ID : " + emp[i].getEmployeeId());
119         System.out.println("Name : " + emp[i].getEmployeeName());
120         System.out.println("Salary : " + emp[i].calculateSalary());
121
122         found = true;
123         break;
```

```
122         found = true;
123         break;
124     }
125 }
126
127 if(!found)
128 {
129     System.out.println("Employee Not Found");
130 }
131 }
132 }
133
134 class Employee
135 {
136     private int employeeId;
137     private String employeeName;
138     private String department;
139     private double basicSalary;
140
141     public Employee()
142     {
143     }
144
145     public Employee(int employeeId, String employeeName,
146                     String department, double basicSalary)
147     {
148         this.employeeId = employeeId;
149         this.employeeName = employeeName;
150         this.department = department;
151         this.basicSalary = basicSalary;
152     }
153
154     public int getEmployeeId()
155     {
156         return employeeId;
157     }
158
159     public String getEmployeeName()
160     {
161         return employeeName;
162     }
163
164     public String getDepartment()
165     {
166         return department;
167     }
168
169     public double getBasicSalary()
170     {
171         return basicSalary;
172     }
173
174     public double calculateSalary()
175     {
176         return basicSalary;
177     }
178
179     public void displayEmployee()
180     {
181         System.out.println("Employee ID : " + employeeId);
182         System.out.println("Name : " + employeeName);
183         System.out.println("Department : " + department);
184     }
185 }
```

```

186 }
187 class TeachingFaculty extends Employee
188 {
189     private int teachingHours;
190     private double incentivePerHour;
191
192     public TeachingFaculty()
193     {
194
195     }
196
197     public TeachingFaculty(int employeeId,String employeeName,
198                           String department,double basicSalary,
199                           int teachingHours,double incentivePerHour)
200     {
201         super(employeeId,employeeName,department,basicSalary);
202         this.teachingHours = teachingHours;
203         this.incentivePerHour = incentivePerHour;
204     }
205
206     @Override
207     public double calculateSalary()
208     {
209         return getBasicSalary() + (teachingHours * incentivePerHour);
210     }
211 }
212
213 class TechnicalStaff extends Employee
214 {
215     private int overtimeHours;
216     private double overtimeRate;
217
218     public TechnicalStaff()
219     {
220
221     }
222
223     public TechnicalStaff(int employeeId,String employeeName,
224                           String department,double basicSalary,
225                           int overtimeHours,double overtimeRate)
226     {
227         super(employeeId,employeeName,department,basicSalary);
228         this.overtimeHours = overtimeHours;
229         this.overtimeRate = overtimeRate;
230     }
231
232     @Override
233     public double calculateSalary()
234     {
235         return getBasicSalary() + (overtimeHours * overtimeRate);
236     }
237 }
238
239 class VisitingFaculty extends Employee
240 {
241     private int lecturesHandled;
242     private double paymentPerLecture;
243
244     public VisitingFaculty()
245     {
246
247     }

```

```
210 /
211 }
212
213 class TechnicalStaff extends Employee
214 {
215     private int overtimeHours;
216     private double overtimeRate;
217
218     public TechnicalStaff()
219     {
220
221     }
222
223     public TechnicalStaff(int employeeId,String employeeName,
224                           String department,double basicSalary,
225                           int overtimeHours,double overtimeRate)
226     {
227         super(employeeId,employeeName,department,basicSalary);
228         this.overtimeHours = overtimeHours;
229         this.overtimeRate = overtimeRate;
230     }
231
232     @Override
233     public double calculateSalary()
234     {
235         return getBasicSalary() + (overtimeHours * overtimeRate);
236     }
237 }
238
239 class VisitingFaculty extends Employee
240 {
241     private int lecturesHandled;
242     private double paymentPerLecture;
243
244     public VisitingFaculty()
245     {
246
247     }
248
249     public VisitingFaculty(int employeeId,String employeeName,
250                           String department,double basicSalary,
251                           int lecturesHandled,double paymentPerLecture)
252     {
253         super(employeeId,employeeName,department,basicSalary);
254         this.lecturesHandled = lecturesHandled;
255         this.paymentPerLecture = paymentPerLecture;
256     }
257
258     @Override
259     public double calculateSalary()
260     {
261         return getBasicSalary() + (lecturesHandled * paymentPerLecture);
262     }
263 }
```

## QUESTIONS

Q1

Smart University Payroll  
Management System  
25 marks

Q2

Smart Vehicle Insurance Claim  
System  
25 marks

2/2 answered

## Q2 Smart Vehicle Insurance Claim System

25 marks

Problem Statement An insurance company processes claims for three categories of vehicles: Car, Bike, and Truck. The claim amount depends on the vehicle type. Requirements Superclass: Vehicle Private members: vehicleNumber ownerName insuredAmount Subclasses: Car Bike Truck Each subclass overrides calculateClaimAmount() using different formulas. The application should: Read details for N vehicles Store them using an array of superclass references Calculate claim amounts using runtime polymorphism Search by vehicle number Display the vehicle with the highest claim amount Question 3 (Very Hard) Online Shopping Order Processing System (50 Marks) Problem Statement An e-commerce company sells products in three categories: Electronics Clothing Grocery The billing process varies for each category. Requirements Superclass: Product Private members: productId productName price quantity Subclasses: Electronics Clothing Grocery Each subclass overrides calculateBill(). The program should: Store products in an array of superclass references Calculate the final bill using runtime polymorphism Search products by ID Display the most expensive order Compute total revenue

Claim Formula (Recommended)

Car

 $\text{Claim} = \text{insuredAmount} \times 0.80$ 

Bike

 $\text{Claim} = \text{insuredAmount} \times 0.60$ 

Truck

 $\text{Claim} = \text{insuredAmount} \times 0.90$ 

Test Case 1

Input

3

Car

TN10AB1234

Ramesh

800000

Bike

TN09XY5678

Suresh

100000

Truck

TN11TR9999

Logan

1500000

Search Vehicle:

TN09XY5678

Expected Output

Vehicle : TN10AB1234

Claim Amount : 640000.0

Vehicle : TN09XY5678

Claim Amount : 60000.0

Vehicle : TN11TR9999

Claim Amount : 1350000.0

Highest Claim Vehicle

TN11TR9999

1350000.0

## Your Code

```
1 import java.util.*;
2
3 class Main
4 {
5     public static String readInput(Scanner sc)
6     {
7         String s="";
8
9         while(sc.hasNextLine())
10        {
11            s=sc.nextLine().trim();
12
13            if(!s.isEmpty())
14                return s;
15        }
16
17        return s;
18    }
19
20    public static void main(String args[])
21    {
22        Scanner sc=new Scanner(System.in);
23
24        int n=Integer.parseInt(readInput(sc));
25
26        Vehicle vehicle[]=new Vehicle[n];
27
28        for(int i=0;i<n;i++)
29        {
30            String type=readInput(sc);
31
32            String vehicleNumber=readInput(sc);
33            String ownerName=readInput(sc);
34            double insuredAmount=Double.parseDouble(readInput(sc));
35
36            if(type.equalsIgnoreCase("Car"))
37            {
38                vehicle[i]=new Car(vehicleNumber,ownerName,insuredAmount);
39            }
40            else if(type.equalsIgnoreCase("Bike"))
41            {
42                vehicle[i]=new Bike(vehicleNumber,ownerName,insuredAmount);
43            }
44            else if(type.equalsIgnoreCase("Truck"))
45            {
46                vehicle[i]=new Truck(vehicleNumber,ownerName,insuredAmount);
47            }
48        }
49
50        String search=readInput(sc);
51
52        if(search.equalsIgnoreCase("Search Vehicle:"))
53        {
54            search=readInput(sc);
55        }
56
57        Vehicle highest=vehicle[0];
58
59        for(int i=0;i<n;i++)
60        {
```

## Input

3  
Car  
TN10AB1234  
Ramesh

## Output

Vehicle : TN10AB1234  
Claim Amount : 640000.0  
  
Vehicle : TN09KYS678  
Claim Amount : 600000.0



```
60     {
61         if(vehicle[i].calculateClaimAmount()>highest.calculateClaimAmount())
62         {
63             highest=vehicle[i];
64         }
65     }
66
67     for(int i=0;i<n;i++)
68     {
69         System.out.println("Vehicle : " + vehicle[i].getVehicleNumber());
70         System.out.println("Claim Amount : " + vehicle[i].calculateClaimAmount());
71         System.out.println();
72     }
73
74     System.out.println("Highest Claim Vehicle");
75     System.out.println(highest.getVehicleNumber());
76     System.out.println(highest.calculateClaimAmount());
77     System.out.println();
78
79     boolean found=false;
80
81     for(int i=0;i<n;i++)
82     {
83         if(vehicle[i].getVehicleNumber().equalsIgnoreCase(search))
84         {
85             System.out.println("Vehicle Found");
86             System.out.println("Owner : " + vehicle[i].getOwnerName());
87             System.out.println("Claim Amount : " + vehicle[i].calculateClaimAmount());
88             System.out.println();
89             found=true;
90             break;
91         }
92     }
93
94     if(!found)
95     {
96         System.out.println("Vehicle Not Found");
97     }
98 }
99 }
100
101 class Vehicle
102 {
103     private String vehicleNumber;
104     private String ownerName;
105     private double insuredAmount;
106
107     public Vehicle()
108     {
109     }
110
111     public Vehicle(String vehicleNumber,String ownerName,double insuredAmount)
112     {
113         this.vehicleNumber=vehicleNumber;
114         this.ownerName=ownerName;
115         this.insuredAmount=insuredAmount;
116     }
117
118     public String getVehicleNumber()
119     {
120         return vehicleNumber;
121     }
122 }
```

```
122     }
123
124     public String getOwnerName()
125     {
126         return ownerName;
127     }
128
129     public double getInsuredAmount()
130     {
131         return insuredAmount;
132     }
133
134     public double calculateClaimAmount()
135     {
136         return insuredAmount;
137     }
138 }
139 class Car extends Vehicle
140 {
141     public Car()
142     {
143
144     }
145
146     public Car(String vehicleNumber,String ownerName,double insuredAmount)
147     {
148         super(vehicleNumber,ownerName,insuredAmount);
149     }
150
151     @Override
152     public double calculateClaimAmount()
153     {
154         return getInsuredAmount() * 0.80;
155     }
156 }
157
158 class Bike extends Vehicle
159 {
160     public Bike()
161     {
162
163     }
164
165     public Bike(String vehicleNumber,String ownerName,double insuredAmount)
166     {
167         super(vehicleNumber,ownerName,insuredAmount);
168     }
169
170     @Override
171     public double calculateClaimAmount()
172     {
173         return getInsuredAmount() * 0.60;
174     }
175 }
176
177 class Truck extends Vehicle
178 {
179     public Truck()
180     {
181
182     }
183
184     public Truck(String vehicleNumber,String ownerName,double insuredAmount)
185     {
186         super(vehicleNumber,ownerName,insuredAmount);
187     }
188
189     @Override
190     public double calculateClaimAmount()
191     {
192         return getInsuredAmount() * 0.40;
193     }
194 }
```

```
184     public Truck(String vehicleNumber, String ownerName, double insuredAmount) {
185     {
186         super(vehicleNumber, ownerName, insuredAmount);
187     }
188
189     @Override
190     public double calculateClaimAmount()
191     {
192         return getInsuredAmount() * 0.90;
193     }
194 }
```

✔ Submitted